Echtzeit-Analyse von IoT-Sensordaten

Nachrichtenbasierte Verarbeitung mit RabbitMQ

API-Design und Systemintegration

Python · Pika · RabbitMQ

Architektur und Nachrichtenfluss

Nachrichtenfluss	Exchange	Ziel-Queue
Producer sendet Messwerte	iot.events	analysis.climate analysis.pressure
Analyse sendet Alarme	alerts	alarms
Analyse sendet Telemetrie	monitoring	monitoring.events
Ablehnung, Ablauf oder Zustelllimit	iot.dlx	iot.dlq

Analyse: climate für Temperatur und Feuchte, pressure für Druck

Routing und Exchange-Auswahl

Einsatz	Entscheidung	Alternative und Abwägung
Sensordaten	topic mit sensor.<type>.<id>	direct benötigt exakte Schlüssel je Sensor
Alarme	direct mit WARNING / CRITICAL	headers ist für zwei Stufen aufwendiger
Monitoring	topic mit monitor.#	fanout erlaubt keine Musterauswahl
Dead Letters	fanout für eine gemeinsame DLQ	direct erlaubt getrennte Fehlerpfade

climate: sensor.temperature.* und sensor.humidity.*
pressure: sensor.pressure.*

Producer: Messwerte zuverlässig senden

Temperatur, Luftfeuchtigkeit und Druck, jeweils im 2-Sekunden-Takt

Beispielnachricht

```
{
  "message_id": "eindeutige UUID",
  "sensor_id": "temp-01",
  "sensor_type": "temperature",
  "timestamp": "2026-10-25T10:00:00",
  "value": 84.2
}
```

Zuverlässige Übertragung

delivery_mode=2

Persistente Nachricht in einer dauerhaften Queue

Retry mit Exponential Backoff

1 s 2 s 4 s 8 s 10 s

Analyse in 10-Sekunden-Fenstern

Validierung vor der Verarbeitung

JSON-Objekt, Pflichtfelder, zulässiger Sensortyp, endlicher Zahlenwert und ISO-Zeitstempel.

Aggregation je Sensortyp

Gültige Werte sammeln. Nach ungefähr 10 Sekunden Anzahl, Mittelwert, Minimum und Maximum ausgeben.

Zeitbezug

10-Sekunden-Fenster nach Empfang beim Consumer

Alarmierung und Priorität

Sensortyp	WARNING	CRITICAL
Temperatur	> 80 °C	> 100 °C
Luftfeuchtigkeit	> 90 %	> 98 %
Druck	< 920 oder > 1080 hPa	< 910 oder > 1090 hPa

Priorität wirkt auf die nächste Auslieferung aus dem Rückstau.

ACK/NACK: Verarbeitung und Fehlerbehandlung

Auszug aus handle(), Alarmversand und Protokollierung gekürzt

```
tag = method.delivery_tag
try:
    msg = validate(body, profile["types"])
    # ... Anomalieprüfung und Alarmversand ...
    window.add(msg["type"], msg["value"])
except InvalidMessage:
    ch.basic_nack(tag, requeue=False)
    return
except pika.exceptions.AMQPConnectionError:
    raise
except Exception:
    ch.basic_nack(tag, requeue=not method.redelivered)
    return
ch.basic_ack(tag)
```

Ungültiger Inhalt

NACK ohne Requeue: direkt über DLX in die DLQ

Verarbeitungsfehler

Erstmals: Requeue
Erneut: DLQ

Erfolgreich

ACK nach Abschluss der Verarbeitung

TTL und Dead Letter Queue

Auslöser	Broker-Verhalten
Ungültige Nachricht oder Retry ausgeschöpft	NACK ohne Requeue leitet an iot.dlx weiter
Nachricht wartet länger als 60 Sekunden	TTL-Ablauf leitet an iot.dlx weiter
Zustelllimit der Analysequeue überschritten	Quorum-Queue leitet an iot.dlx weiter

Einordnung

Veraltete Messwerte verlieren ihren Echtzeitnutzen. iot.dlq hält ausgesonderte Nachrichten zur Untersuchung bereit.

Monitoring über RabbitMQ

1. Analyse veröffentlicht Ereignisse

Exchange: monitoring (topic)
Routing Keys: monitor.processed und monitor.failed

2. Binding leitet an die Queue weiter

Binding-Muster: monitor.#
Ziel-Queue: monitoring.events

3. Monitoring-Consumer liest die Queue

Er liest aus monitoring.events und gibt alle fünf Sekunden Verarbeitungszahlen, Alarme und die durchschnittliche Analysezeit aus.

Live-Demo: regulärer Betrieb

Fehlerszenario 1: Alarm-Consumer fällt aus

Schritt	Dokumentiertes Verhalten und Einordnung
Auslöser	Demo-Skript: kill -9 nach x s, Neustart nach y s
Nachrichten und ACK / NACK	Wartende bleiben in der Queue. Kein ACK/NACK beim Crash.
Erneute Zustellung	Alarme bleiben bis zum ACK in der Queue.
Wiederanlauf	Consumer verarbeitet erneut, CRITICAL erhält Vorrang.
DLQ-Bedingung	Alarme mit abgelaufener Queue-TTL gelangen in die DLQ.

Fehlerszenario 2: falscher Sensortyp

```
python faultyProducer.py
```

Schritt	Dokumentiertes Verhalten und Einordnung
Auslöser	Fehlerhafter Producer sendet einen falschen Sensortyp.
Validierung	validate() erkennt einen im Analyseprofil unzulässigen Typ.
ACK / NACK und Retry	NACK mit requeue=False, keine erneute Zustellung.
DLQ und Weiterbetrieb	iot.dlx leitet zur iot.dlq. Gültige Daten laufen weiter.

Erkenntnis: Ein dauerhafter Inhaltsfehler braucht keinen weiteren Versuch.

Erkenntnisse aus Umsetzung und Fehlerszenarien

ACKs machen Wiederanlauf möglich

Ein Consumer-Absturz beendet die Verarbeitung, ohne die Nachricht zu bestätigen. Der Broker kann sie erneut zustellen.

Fehlerart bestimmt den weiteren Weg

Vorübergehende Ausfälle erlauben eine erneute Verarbeitung. Dauerhaft ungültige Nachrichten gelangen direkt in die DLQ.

Logging erleichtert die Fehleranalyse

print-Ausgaben können gepuffert und verzögert erscheinen. Logging mit StreamHandler liefert zeitnahe Meldungen mit Zeitstempel und Log-Level.

Technische Grenzen des aktuellen Stands

Aktueller Stand	Auswirkung	Verbesserung
Prototyp mit simulierten Sensordaten	Verhalten unter realer Last bleibt offen.	Reale Sensoren anbinden und Lasttests durchführen.
Zeitfenster nach Empfangszeit	Verzögerte Messungen landen in späteren Fenstern.	Ereigniszeit und verspätete Daten berücksichtigen.
Feste Queue-TTL von 60 Sekunden	Längerer Rückstau führt zu abgelaufenen Daten.	TTL und Verarbeitungskapazität gemeinsam abstimmen.